

MCP Integration Patterns & Advanced Tool Boundaries

1. Tool namespacing — solving multi-server collisions

When Claude connects to multiple MCP servers simultaneously, tool name collisions occur when two servers expose tools with the same name. Claude cannot reliably distinguish them and behaves unpredictably. Tool namespacing solves this by prefixing every tool name with its server name.

Without namespacing ✗	With namespacing ✓
GitHub: list_issues	GitHub: github_list_issues
Jira: list_issues	Jira: jira_list_issues
Slack: list_channels	Slack: slack_list_channels
DB: list_records	DB: db_list_records
<i>Claude picks unpredictably between the two list_issues tools.</i>	<i>Claude can now clearly distinguish all tools by server origin.</i>

```
// Namespaced tool definition in MCP server config
{
  "server_name": "github",
  "tool_prefix": "github_",
  "tools": [
    { "name": "github_list_issues", "description": "..." },
    { "name": "github_create_pr", "description": "..." },
    { "name": "github_merge_branch", "description": "..." }
  ]
}
```

Namespacing — exam rules

- Format: {server_name}_{tool_name} — always lowercase, underscore separator.
- Namespacing is a DISAMBIGUATION pattern, not a security feature.
- Exam trigger: 'Two MCP servers with tools of the same name' → answer is namespacing.
- Descriptions should also reference the server: 'Lists issues from GitHub (not Jira)'.
- Applied at the server level — all tools in a server share the same prefix.

2. Schema versioning — safe vs breaking changes

MCP servers need versioning strategies when tool schemas change. A breaking schema change without versioning breaks all existing Claude integrations. The key rule: more permissive = safe. More restrictive = breaking.

Change type	Example	Breaking?	Action needed
Remove required field	<i>customer_id removed from schema</i>	✗ BREAKING	Version the tool
Rename required field	<i>user_id renamed to id</i>	✗ BREAKING	Version the tool
Change field type	<i>integer changed to string</i>	✗ BREAKING	Version the tool
Add new required field	<i>New mandatory email field added</i>	✗ BREAKING	Version the tool
Make required → optional	<i>customer_id made optional</i>	✓ SAFE	Modify in-place
Add new optional field	<i>New optional notes field added</i>	✓ SAFE	Modify in-place
Add new tool entirely	<i>New get_customer_v2 tool added</i>	✓ SAFE	Add to server

The permissiveness rule — memory hook

- More permissive (loosening constraints) = SAFE change.
- More restrictive (tightening constraints) = BREAKING change.
- Required → Optional = loosening = SAFE.
- Optional → Required = tightening = BREAKING.
- Adding optional field = loosening = SAFE.
- Removing any field = tightening = BREAKING.
- Breaking changes need a VERSIONED tool — do not modify in-place.

3. MCP error handling — three types, three responses

MCP tool calls can fail in three distinct ways. Each requires a different response. The exam tests whether you can identify the error type and choose the correct strategy.

Error type	Example	Retry?	Response
Schema validation failure	<i>Missing required field, wrong type</i>	Yes — after fixing input	Self-correct parameters, retry once
Transient failure	<i>Rate limit, timeout, 503 unavailable</i>	Yes — with backoff	Exponential backoff, cap at 2-3 attempts
Permanent failure	<i>AUTH_FAILED, 404 not found, permission denied</i>	Never	Fallback or escalate immediately

4. The read/write split — most important boundary pattern

Always separate read operations from write operations into distinct tools, even if they touch the same data. This is the most tested boundary pattern in the MCP domain.

Wrong ✗	Right ✓
manage_records (reads AND writes)	query_records (read-only) + update_record (write)
handle_emails (reads AND sends)	read_email + send_email
database_tool (all operations)	select_data + insert_data + update_data + delete_data
process_payment (validates + charges)	validate_payment + charge_payment

Why the read/write split matters

- Enforces minimal footprint at the tool level.
- Claude never has write access when it only needs to read.
- A prompt injection in a read tool cannot cause writes if write is a separate tool.
- Enables human-in-the-loop checkpoints between read and write operations.
- Each tool is independently auditable and testable.

5. Tool composition vs monolithic tools

Tool composition means building complex workflows by chaining simple tools — each tool does one step, Claude orchestrates them. Monolithic tools try to do everything in one call — they create boundary violations.

```
// Monolithic – boundary violation
process_order(order_id)
→ validates + charges card + updates inventory + sends email + logs audit

// Composed – correct design
validate_order(order_id)
→ charge_payment(order_id, amount) // pause here for human confirmation
→ update_inventory(product_id, quantity)
→ send_confirmation_email(order_id)
→ log_audit_event(order_id, action)
```

Why composition wins

- Each step is independently auditable.
- Reversible at the right point — can pause before irreversible steps (charging a card).
- Human-in-the-loop confirmation possible between steps.
- Each tool is independently testable and replaceable.
- Monolithic tools make it impossible to pause before irreversible actions.

6. Tool permission matrix

Each tool's permissions are exactly what it needs — nothing more. Use this framework when the exam asks which permissions a tool should have.

Tool type	Read data?	Write data?	Delete data?	Send external?
Query / lookup	✓	✗	✗	✗
Create / insert	✓	✓ (insert only)	✗	✗
Update	✓	✓ (update only)	✗	✗
Delete	✓	✗	✓	✗
Notify / send	✗	✗	✗	✓

7. Quick recall — 3 things to remember

- Namespacing solves tool name collisions.**
 Format: {server}_{tool}. It is a disambiguation pattern, not a security feature. Exam trigger: two servers with same tool name.
- Breaking change = more restrictive. Safe change = more permissive.**
 Required → Optional = safe. Optional → Required = breaking. Adding optional fields = safe. Removing fields = breaking. Breaking changes need versioned tools.
- Three error types, three responses.**
 Schema error → fix and retry. Transient → backoff and retry. Permanent → fallback or escalate. NEVER retry a permanent failure.

8. Practice questions & answers

Three questions from today's session. Q2 was answered incorrectly — the permissiveness rule is a frequent exam trap. Review the explanation carefully.

Q1. Claude is connected to both a GitHub MCP server and a Jira MCP server. Both expose a tool called `list_issues`. Claude behaves unpredictably — sometimes querying GitHub, sometimes Jira. What is the root cause and correct fix?

- A) Conflicting authentication credentials — fix by using separate API keys
- B) Claude cannot connect to more than one MCP server at a time — remove one
- ✓ **C) Tool name collision — fix by namespacing tools with server prefixes (`github_list_issues`, `jira_list_issues`)**
- D) Tool descriptions are too similar — fix by rewriting one to be more distinct

Explanation:

- C is correct: When two MCP servers expose tools with the same name, Claude cannot reliably distinguish them. Namespacing prefixes every tool with its server name — `github_list_issues` and `jira_list_issues` are now unambiguous.
- A is wrong: Authentication is per-server. Separate API keys don't solve naming collision.
- B is wrong: Claude absolutely supports multiple MCP servers simultaneously.
- D is wrong: Better descriptions help but don't solve the root cause. Two tools with the same name is architecturally broken. Namespacing is the structural fix.

Q2. An MCP server exposes `get_customer` with schema: `{required: ['customer_id']}`. The team wants to also accept email as an alternative lookup key. They propose making `customer_id` optional. What is the risk?

Your answer: B

✓ **A) No risk — making a required field optional is backward-compatible and existing integrations continue working**

- B) Making `customer_id` optional is a breaking change — existing Claude prompts that rely on it being required will fail
- C) Making `customer_id` optional is safe, but they should version the tool anyway for audit purposes
- D) This is a permanent failure scenario — the tool must be deleted and recreated

Explanation:

- A is correct: Making a required field optional is MORE PERMISSIVE — a backward-compatible change. Existing prompts that pass `customer_id` still work. The schema accepts more, not less.
- B is wrong (the trap): You correctly sensed risk but misidentified the direction. Making a field optional loosens the constraint — that is safe. Making an optional field required would be the breaking change.
- The permissiveness rule: More permissive = safe. More restrictive = breaking. Required → Optional = loosening = SAFE.
- C is wrong: Versioning for audit alone is over-engineering. Version only for genuine breaking changes.
- D is wrong: Permanent failure is a runtime error concept, not a schema design concept.

Q3. A Claude agent calls MCP tool `send_notification`. The tool returns: `{error: 'AUTH_FAILED', message: 'Invalid API key for notification service', retryable: false}`. What should the agent do?

- A) Retry with exponential backoff — all network errors resolve eventually
- B) Retry once immediately, then wait 30 seconds and retry again
- ✓ C) **Do not retry — permanent failure. Fall back to alternative or escalate to orchestrator**
- D) Retry up to 3 times with backoff, then report failure to the user

Explanation:

- C is correct: `AUTH_FAILED` with `retryable: false` is a PERMANENT failure. An invalid API key will not resolve itself. Retrying wastes time. Fall back to alternative or escalate.
- A is wrong: 'All network errors resolve eventually' is false. Auth failures, missing resources, and permission errors are permanent — they require human intervention.
- B and D are wrong: Backoff patterns apply to TRANSIENT errors (rate limits, timeouts). The `retryable: false` flag and `AUTH_FAILED` type definitively indicate permanent failure.
- Error type rule: Schema error → fix and retry. Transient → backoff. Permanent → fallback. Never retry permanent failures.

Day 9 scorecard

2 / 3

Good session — permissiveness rule to lock in

Tool name collision & namespacing	✓ Correct
Breaking vs backward-compatible schema changes	✗ Review
Permanent failure — no retry	✓ Correct

Key takeaway from missed Q2 — the permissiveness rule

- More permissive (loosening) = SAFE change. More restrictive (tightening) = BREAKING.
- Required → Optional = loosening = SAFE.
- Optional → Required = tightening = BREAKING.
- Adding optional field = loosening = SAFE.
- Removing any field = tightening = BREAKING.
- Exam trap: 'Making customer_id optional' sounds risky but is actually safe — it loosens the schema.
- Breaking changes need VERSIONED tools. Safe changes can be modified in-place.

Next up: Day 10

Week 2 Friday quiz — 10-question mixed review of Claude Code + MCP domain (38% of exam).